

(1) Overview

GEEODE: A Google Earth Engine Implementation of Optimization by Differential Evolution

Paper Authors

1. Routh, Devin
2. Rösli, Claudia

Paper Author Roles and Affiliations

1. Software Developer; Remote Sensing Laboratories, Department of Geography, University of Zürich, Zürich, Switzerland
2. Principal Investigator and Lead Scientist; Remote Sensing Laboratories, Department of Geography, University of Zürich, Zürich, Switzerland

Abstract

This function module was written for Google Earth Engine (GEE) as an implementation of the Differential Evolution algorithm for optimizing functions (i.e., fitting curves) on remotely sensed imagery data. Its purpose is to allow the user to fit any arbitrary supported closed-form algebraic functions (expressible using the GEE expression syntax) on GEE's image collection objects, making the module particularly useful for time series analyses on satellite image collections that require flexible curve fitting algorithms (e.g., double-logistic functions with several parameters). The function makes extensive use of the array image object, which embeds multidimensional arrays at every pixel and thus allows for matrix calculations using pixel level time series matrices. Moreover, the module was designed to produce a variety of outputs: either a final value (i.e., an optimized set of parameters that can be used to fit a functional form or curve to the image collection data) or intermediate values called “populations”. Populations are lists of candidate parameters that are being considered in the algorithm for fitting a curve to the image collection data. The option to produce intermediate outputs in the form of full populations allows users to run analyses in series to refine the best fit possible, referred to as a “daisy-chain” analysis when done in repetition. Moreover, producing intermediate outputs also allows users to run multiple populations in parallel. The function is implemented in both Javascript and Python, and an example on Sentinel-2 data time series is included.

Keywords

Google Earth Engine; differential evolution; remote sensing; time series analysis; curve fitting; evolutionary optimization;

Introduction

Optimization algorithms based on natural selection, also called genetic or evolutionary algorithms, have been used since the 1950s [10] and continue to be re-examined for use as well as development [1, 4, 5, 12, 13]. The idea is simple: iterate a population of candidate solutions to a problem while mixing candidate solutions in the same ways that populations of organisms undergo genetic variation. For example, if you have observation points in 2 dimensions (see Figure 1 below), and you need to fit a specific mathematical model (e.g., a logarithmic function including 3 parameters a , b , and c), the algorithm proceeds by (1) randomly generating a population of candidate mathematical models that are fit to the data (2) then undergoing an “evolution” process where you mix/combine best-fitting models, iteratively, until a satisfactory model has been reached.

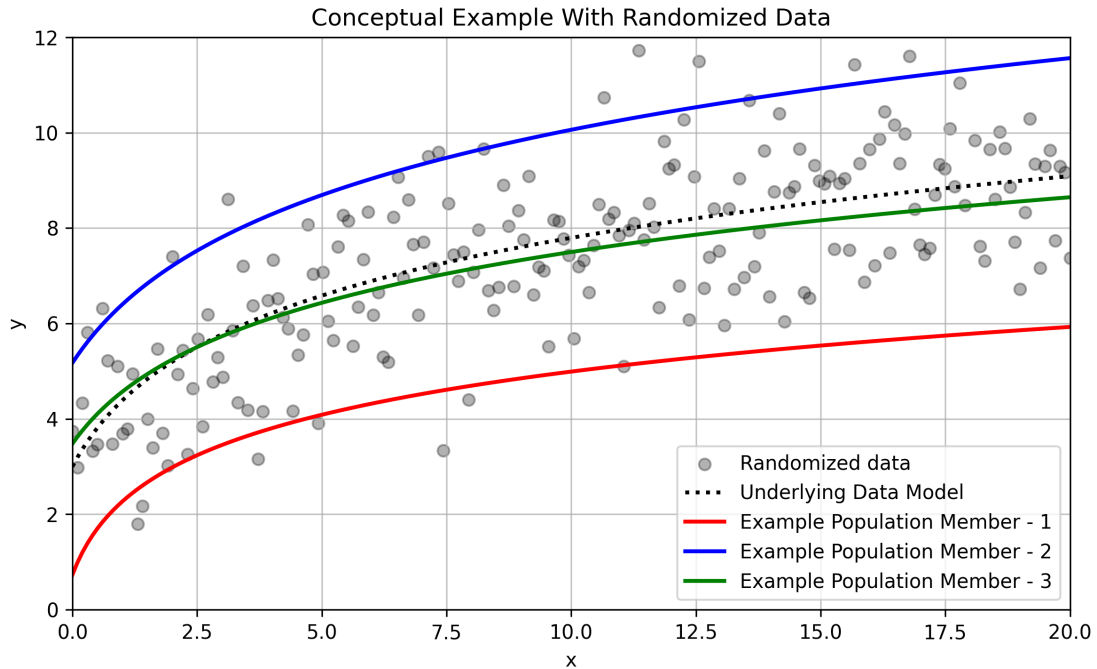


Figure 1: DE Optimization in a single chart

Storn and Price [16] developed such an algorithm that they called differential evolution (DE) that is particularly suited for optimizing non-continuous and non-differentiable real-valued functions using real-valued parameters, and the approach has since been applied in a number of scholarly fields [2, 6, 14, 15]. Nowadays, there are multiple software packages available to run DE, including a package in R [11], a function in SciPy[17], and commercial tools in MatLab[8] and Mathematica[9].

The implementation of DE in GEE aims to assist with modelling time series data using arbitrary mathematical models (e.g., linear, parabolic, logarithmic, etc.) on global-scale remote-sensing datasets—especially when single pixels have sparse observations, such as when remotely sensed data is heavily impeded by cloud cover, and the process to fit a model through more standard regression modelling ap-

proaches are unsuitable. Moreover, the algorithm is structured to allow the production of intermediate outputs (in the form of populations of optimized models, rather than just a final optimized model), which allows users to take advantage of GEE’s queueing system and gives greater options for task-based parallelization to accomplish computationally rigorous workflows on large amounts of imagery.

Implementation and architecture

When searching for optimal parameter values, the algorithm benefits from a higher number of iterations in addition to a greater number of potential population members; i.e., optimization will improve when the algorithm is (1) testing more potential population candidates and (2) taking a greater number of iteration steps to improve these potential options many times. Both a higher number of population members, as well as a greater number of iterations, require greater computational memory and resources. This implementation was structured accordingly to parallelize computation as much as possible while also making it possible to iteratively produce intermediary populations as evolution progresses.

More specifically, if users hit memory limits with their population number or their number of iterations, the algorithm allows users to structure their workflow via a divide-and-conquer approach: the users may run any number of populations independently of one another then combine the optimal parameter vectors from every separate population run into a final population that can be further iterated. Dividing the populations in this way allows users to run multiple sub-populations in parallel as Earth Engine tasks, with each using the maximum amount of memory possible for a single task.

This means the maximum population size possible for this function is the maximum number of population candidates that can be run via a single Google Earth Engine task for 1 iteration; i.e., a job where all memory available is used for population size while still making a single iteration. In this case, a population of mutated vectors from a previous iteration is used as the input into a further iteration into a follow-up Google Earth Engine task. It’s for this reason that the implementation allows the export of full populations, in the form of GEE array-images, rather than just single model parameter sets. It allows users to follow the “daisy-chain” procedure with the output of one iteration becoming the input into the following iteration.

The algorithm is furthermore structured to produce metrics that help the user decide when their time series model has been optimized to a desired degree of fitness; i.e., when a chosen RMSE value has been achieved. The implementation includes an option to produce an RMSE image, termed a *screeImage*, to monitor the progression of the optimization success with a scree plot similar to what is used in dimensional reduction techniques [3]. This image is comprised of multiple bands, wherein each band is the best RMSE value from the population at that iteration. It allows users to determine when convergence on an optimal value has been achieved and an acceptable final parameter set has been produced.

In addition to the core functionality provided by the `de_optim` function, specific

functions have been provided (both analytical and practical) to allow users the ability to customize their workflows.

Specifically, given the constraints on memory that affect the parameterization of the optimization process, a temporal subsampling function has been added to give users the ability to reduce the size of the image collections being used as the basis for their optimization tasks.

The subsampling function specifically allows for a temporal subsetting process, in which every pixel's time-series of observations is sampled according to their temporal density; i.e., time series are reduced to a specific size by removing the necessary number points at an individual pixel-level according to their relative frequency across the timespan through a weighted-random sampling process wherein the weights are then calculated as the density of observations around a specified time-window/kernel. See the documentation for more details.

Quality control

Included in the repository is also a PyTest-based testing framework to confirm the correct operation of the algorithm. It works by asserting specific algorithmic conditions when tested across arbitrary functional forms specified with randomly generated parameter sets. In other words, the testing process allows users to confirm that the algorithm:

- **Generates sets of functional coefficients on arbitrary closed-form algebraic functions** such that each coefficient **falls within the numeric bounds** provided.
- Moreover, when the coefficient sets are being assessed **as the iterations proceed**, the fitness score of the coefficients **either improves or stabilizes without exception**.
- And lastly, the iterations' **RMSE should converge on (or towards)**, in this case, **the RMSE of the known randomly-generated function** (within an acceptable buffer).

The existing PyTest framework assesses a variety of functional families—currently including multiple replicates of logarithmic, quadratic, harmonic, and linear functions—while also allowing users a structure to expand on the tests *ad hoc* in order to affirm algorithmic fidelity. Regarding the last point on convergence, the tests successfully demonstrate that larger populations (10+) running over a higher number of iterations (25+) bring the RMSE close to the randomly-generated functions' RMSE; all replicates safely fell within a buffer of is x2 the known RMSE (with most replicates coming much closer or even exceeding the known function's RMSE).

The PyTest framework also includes a test of the package's principal temporal subsetting function. This test uses a known synthetic time series to ensure the final

outputs follow the correct sampling strategies, do not perturb input data, and can accept any temporal date value (including natural zero values).

Running the PyTest requires either existing local GEE Credentials or an authenticated GEE Token set as a shell environment variable (i.e., `EARTHENGINE_TOKEN`). Users must then input the asset root directory of their associated Google Earth Engine account as a string when executing the PyTest command. The tests requires a <1MB of quota space. All aspects, including cleanup, are detailed in the `README`.

(2) Availability

Operating system

For Python, any OS capable of running Python ≥ 3.11 , < 4.0 , the Google Earth Engine API, and Pandas ≥ 2.0 , < 3.0

For Javascript, simply use the Earth Engine code editor [7].

Programming language

Python and Javascript

Additional system requirements

Since computing revolves around the use of Google Earth Engine, the vast majority of the processing is done remotely and therefore requires an operational internet connection.

Dependencies

For Python: Python ≥ 3.11 , < 4.0 ; earthengine-api 0.1.382; Pandas ≥ 2.0 , < 3.0

List of contributors

Noel Gorelick; one of the Google Earth Engine founders and an engineer at Google during the time of his contribution.

Software location:

Name: Zenodo

Persistent identifier: <https://doi.org/10.5281/zenodo.18611213>

Licence: BSD 3-Clause License

Publisher: Devin Routh

Version published: v1.0.1

Date published: 2026-02-11

Code repository

Name: GitHub

Persistent identifier: <https://github.com/uzh-eoas/geeode.git>

Licence: BSD 3-Clause License

Date published: 2025-03-26

Language

English

(3) Reuse potential

While the principal use-case for this module was time series of remote sensing imagery data, *any* arbitrary time series can be input into the function, expanding the possibility for reuse by researchers across a wide range of domains given the scale of Google Earth Engine data catalog (in addition to the fact that users can upload their own data) [7].

At the time of publication, the current mutation functions and the crossover functions are coded in an attempted modularized fashion so additional functions can be developed in the future. The current default mutation option *rand* randomly selects three parameter vectors from the population, computes the difference between two of the vectors, multiplies the difference by a scale factor (a real number between 0 and 2), then adds it to the third randomly chosen vector. The other available mutation function (*best*) performs the same arithmetic except it uses the best parameter vector (according to RMSE) instead of a third randomly chosen vector. The crossover function is a simple binomial whereby each iteration is tested according to whether a random number drawn from a uniform distribution on $[0, 1)$ exceeds a user-supplied crossover value in the range $(0, 1)$.

Ongoing and future developments to the code include a helper function to randomly subsample dense input time series according to relative temporal density, allowing greater control of input size so that memory limits can be better bypassed, as well as potential variations of the crossover function. Moreover, sampling methodologies for the temporal subsetting function can be expanded according to any statistical method required.

Contributors and researchers can submit then discuss issues via the GitHub repository. Contact information for those responsible for the code are also provided on both the repo and the associated documentation.

Acknowledgements

We would like to express gratitude to the engineers behind Google Earth Engine as well as the stewards of their data catalog.

Funding statement

We would like to affirm that our research was funded partly by the Canton of Zürich and partly through a Google Earth Engine research award.

Competing interests

There are no competing interests to declare.

References

- [1] Mohamad Faiz Ahmad et al. “Differential evolution: A recent review based on state-of-the-art works”. In: *Alexandria Engineering Journal* 61.5 (2022), pp. 3831–3872. DOI: 10.1016/j.aej.2021.09.013.
- [2] Robin Biesbroek. “A comparison of the differential evolution method with genetic algorithms for orbit optimisation”. In: 57th International Astronautical Congress. 2006, pp. C1–4. DOI: 10.2514/6.iac-06-c1.4.02.
- [3] Raymond B Cattell. “The scree test for the number of factors”. In: *Multivariate behavioral research* 1.2 (1966), pp. 245–276. DOI: 10.1207/s15327906mbr0102_10.
- [4] Swagatam Das, Sankha Subhra Mullick, and Ponnuthurai N Suganthan. “Recent advances in differential evolution—an updated survey”. In: *Swarm and Evolutionary Computation* 27 (2016), pp. 1–30. DOI: 10.1016/j.swevo.2016.01.004.
- [5] Swagatam Das and Ponnuthurai Nagaratnam Suganthan. “Differential evolution: A survey of the state-of-the-art”. In: *IEEE Transactions on Evolutionary Computation* 15.1 (2010), pp. 4–31.
- [6] Vitaliy Feoktistov. *Differential evolution*. Springer, 2006. DOI: 10.1007/978-0-387-36896-2.
- [7] Noel Gorelick et al. “Google Earth Engine: Planetary-scale geospatial analysis for everyone”. In: *Remote Sensing of Environment* 202 (2017), pp. 18–27.
- [8] The MathWorks Inc. *MATLAB version: 9.13.0 (R2022b)*. Natick, Massachusetts, United States, 2022. URL: <https://www.mathworks.com>.
- [9] Wolfram Research Inc. *Mathematica, Version 15.0*. Champaign, IL, 2026. URL: <https://www.wolfram.com/mathematica>.
- [10] Melanie Mitchell. *An introduction to genetic algorithms*. MIT Press, 1998. DOI: 10.7551/mitpress/3927.001.0001.
- [11] Katharine M Mullen et al. “DEoptim: An R package for global optimization by differential evolution”. In: *Journal of Statistical Software* 40 (2011), pp. 1–26. DOI: 10.32614/CRAN.package.DEoptim.
- [12] Millie Pant et al. “Differential Evolution: A review of more than two decades of research”. In: *Engineering Applications of Artificial Intelligence* 90 (2020), p. 103479. DOI: 10.1016/j.engappai.2020.103479.

- [13] Kenneth Price, Rainer M Storn, and Jouni A Lampinen. *Differential evolution: a practical approach to global optimization*. Springer Science & Business Media, 2006. DOI: 10.1007/3-540-31306-0.
- [14] Kenneth V Price et al. “Application of differential evolution to the analysis of X-ray reflectivity data”. In: *Differential Evolution: A Practical Approach to Global Optimization* (2005), pp. 463–478. DOI: 10.1007/3-540-31306-0_16.
- [15] Anyong Qing. “Basics of differential evolution”. In: *Differential Evolution in Electromagnetics*. Springer, 2010, pp. 19–42. DOI: 10.1007/978-3-642-12869-1_2.
- [16] Rainer Storn and Kenneth Price. “Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces”. In: *Journal of Global Optimization* 11 (1997), pp. 341–359. DOI: 10.1023/A:1008202821328.
- [17] Pauli Virtanen et al. “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python”. In: *Nature Methods* 17 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2.